

Sorting Algorithms: Overview and Objectives

Bubble, selection, insertion, merge, and quick sort

1. Overview

Sorting means arranging a collection of items into a specified order, such as smallest to largest. It is one of the oldest and most studied problems in computer science, and it appears constantly in real programs: ordered search results, ranked lists, leaderboards, and data prepared so that a fast search can work at all. The binary search from the algorithm analysis unit only works on sorted data, so sorting is what makes that speedup available.

This unit covers five sorting algorithms. Three are simple and closely related: **bubble sort**, **selection sort**, and **insertion sort**. Two are divide and conquer algorithms: **merge sort** and **quicksort**. Libraries already provide a sort, so it is fair to ask why we write our own. The answer is that each of these five makes a different trade-off between speed, memory, and behavior on different kinds of input, and working through them is the best practice available for reasoning about an algorithm rather than only using one.

This unit is also where two earlier units get used together. Recursion gives merge sort and quicksort their structure, and algorithm analysis gives us the tools to count the work each sort does and compare them fairly.

For every sort, the goal is the same four things: you can **perform** it by hand, **code** it, **analyze** it, and **compare** it with the others. The larger goal is the method itself. Once you can carry those steps through five sorts, you can carry them through a sort you have never seen. By the end of algorithm analysis and sorting together, you should also be able to turn the same method on your own solution to a problem: count what it does, name its growth rate, and decide whether it is good enough.

As in earlier units, we work in **two languages at once**. Computer Science students write **C/C++**; CIS students write **Java**. Each of the five sorts is the same algorithm in both languages, and only the syntax and the way an array is handed to a method differ. Where those differences matter, we call them out.

The most useful preparation is to trace each sort by hand on a small array of five or six numbers before you look at any code. If you can predict the array after every pass, the code is much easier to write and to debug.

2. Learning objectives

By the end of this unit you should be able to:

Remember and understand

- Define sorting, and state what it means for an array to be in ascending or descending order, including how equal values are treated.
- Describe in plain language the central idea of each of the five sorts: bubble sort swaps neighboring items that are out of order, selection sort repeatedly finds the largest item in the unsorted part of the array and swaps it into the last unsorted position, insertion sort grows a sorted region one item at a time, merge sort splits the array and then merges the sorted halves, and quicksort partitions the array around a pivot.
- Define the vocabulary of sorting: pass, comparison, swap, stable, in place, pivot, partition, and merge.
- State the best-case, average-case, and worst-case Big-O class for each of the five sorts.
- Explain, conceptually, what the pivot does in quicksort and why the choice of pivot affects how well the algorithm performs.
- Recognize merge sort and quicksort as divide and conquer algorithms, and name the divide, conquer, and combine parts of each.

Apply

- Trace each of the five sorts by hand on a small array, showing the contents of the array after each pass, split, merge, or partition.
- Write a working implementation of each sort in Java (CIS) or C/C++ (CS), using loops for the three simple sorts and recursion for merge sort and quicksort.
- Count the comparisons and swaps a given sort performs on a specific small input, and check that count against your trace.
- Apply the counting method from algorithm analysis to a sort's loops to find its Big-O class. For example, a loop nested inside a loop over the same array points to $O(n^2)$.

Analyze

- Determine which input produces the best case and which produces the worst case for each sort (for example, an already sorted array, a reverse-ordered array, or a random one), and explain why these are not the same for every sort.
- Explain why bubble sort, selection sort, and insertion sort are $O(n^2)$ by examining how their loops are structured, and identify which of the three does less work on an already sorted array, and why.
- Explain why merge sort is $O(n \log n)$: the array is halved about $\log n$ times, and the merging at each level does about n work in total. Connect the halving to the binary search analysis.
- Explain why quicksort averages $O(n \log n)$ but degrades to $O(n^2)$ in its worst case, in terms of how evenly each partition splits the array.

- Explain why counting comparisons alone can be misleading, and distinguish a sort that does many comparisons but few swaps from one that does the opposite.
- Compare the five sorts on stability, whether they sort in place, and how much extra memory they need.
- Identify the base case, the recursive step, and the point where the real work happens in merge sort and in quicksort, and relate each to the recursion unit.
- Given a sort you have not seen before, such as a variation of one of the five or a new sort described in pseudocode, trace it, decide whether it is correct, and determine its Big-O class using the same steps you used for the five in this unit.

Evaluate

- Choose a sorting algorithm for a described situation (for example, a small array, a nearly sorted array, limited memory, a need for stability, or a need for a guaranteed worst case), and justify the choice against the alternatives.
- Critique a claim such as “quicksort is always the fastest sort” or “this sort is $O(n)$ because it has only one loop,” using best, average, and worst cases and the work hidden inside each loop.
- Judge whether a given sorting implementation is correct and efficient, and identify and explain any error, such as an off-by-one boundary, a missing base case, or unnecessary passes.
- Analyze and judge your own solution to a problem, sorting or otherwise: count what it does, identify its dominant term and its Big-O class, decide whether that is acceptable for the size of input you expect, and propose an improvement if it is not.